

Supplementary Document for ‘GNNs for Class-Incremental Learning on Circuits’

Rupesh Raj Karn, Johann Knechtel, Ozgur Sinanoglu
Center for Cyber Security, New York University, Abu Dhabi, UAE.
Email: {rupesh.k, johann, ozgursin}@nyu.edu

I. EXAMPLE OF HYPERPARAMETER TUNING

Fig. 1 illustrates the hyperparameter tuning for the GCN static architecture. The results clearly indicate that the choice of hyperparameters during the very first batch (e.g., hidden layer size, learning rate, and dropout) plays a crucial role in sustaining the accuracy of the model across subsequent batches. This highlights that careful hyperparameter selection at the beginning of CIL establishes a stable foundation for the learning process.

We report per-batch accuracy rather than per-task accuracy because CIL evaluates performance after each new batch of classes is introduced, reflecting the cumulative effect of learning sequentially. Unlike task-continual learning, the accuracy of prior tasks while training on a new task is not shown, since the focus is on how well the entire model retains knowledge across all previously seen classes at each batch step.

Figure similar to 1 for other GNNs, hyperparameters, and link predictions are available in our release [1].

II. EXPERIMENTAL VARIABILITY AND STATISTICAL SIGNIFICANCE

A. Motivation

The experiments reported in the main paper were obtained from extensive hyperparameter searches and deterministic training pipelines tailored to circuit netlists. This section clarifies how variability was handled and why the reported results are robust.

B. What was done ?

- All reported results derive from the best configurations found by six complementary search strategies: Grid, Random, Bayesian, Hyperband, Population Based Training (PBT), and Evolutionary algorithms. Each search explores many hyperparameter combinations and thus implicitly samples the configuration space.
- For every selected best configuration we preserved full training logs: per-batch accuracies, weight-distance traces, mechanism-specific metrics (EWC Fisher, SI path integrals, replay effectiveness), and optimizer histories. These artifacts are included in the repository and the Jupyter notebooks referenced in the README.

- No additional random-seed sweeps were executed for the camera-ready submission due to compute and time constraints; instead, the search breadth and log inspection were used as robustness proxies.

C. Qualitative assessment of variability

Using the existing logs we performed the following analyses; saved figures and tables are available in the repository notebooks:

- 1) **Per-batch trajectory inspection.** For each best configuration we inspected per-batch accuracy curves and weight-distance traces. Most configurations show smooth, mechanism-consistent trends (e.g., replay stabilizes batch-wise accuracy; EWC/SI reduce parameter drift).
- 2) **Search-derived robustness.** Multiple search strategies converged to similar high-performing configurations for the same mechanism and architecture (for example, Hyperband and Bayesian often selected similar buffer sizes and learning rates for Rehearsal), indicating that the reported configurations are not isolated outliers.
- 3) **Identification of high-variance settings.** Some dynamic settings and a few architecture/mechanism pairs exhibited larger per-batch fluctuations. These cases are explicitly annotated in the notebooks and discussed in the main text as instability due to expansion and parameter drift.

D. Why exhaustive seed sweeps were not performed ?

While multiple independent seeds per best configuration are ideal for formal statistical testing, the following practical considerations guided our approach:

- The hyperparameter searches themselves sample many configurations; convergence of different search strategies to similar configurations provides evidence of robustness.
- The preserved logs permit post-hoc inspection of training dynamics without re-running experiments.
- The circuit datasets used are deterministic netlists with limited stochasticity in data splits and sampling; this reduces but does not eliminate run-to-run variance.

III. SCALABILITY AND COMPUTATIONAL COST

This section provides theoretical complexity, practical considerations, and empirical runtime notes derived from recorded logs.

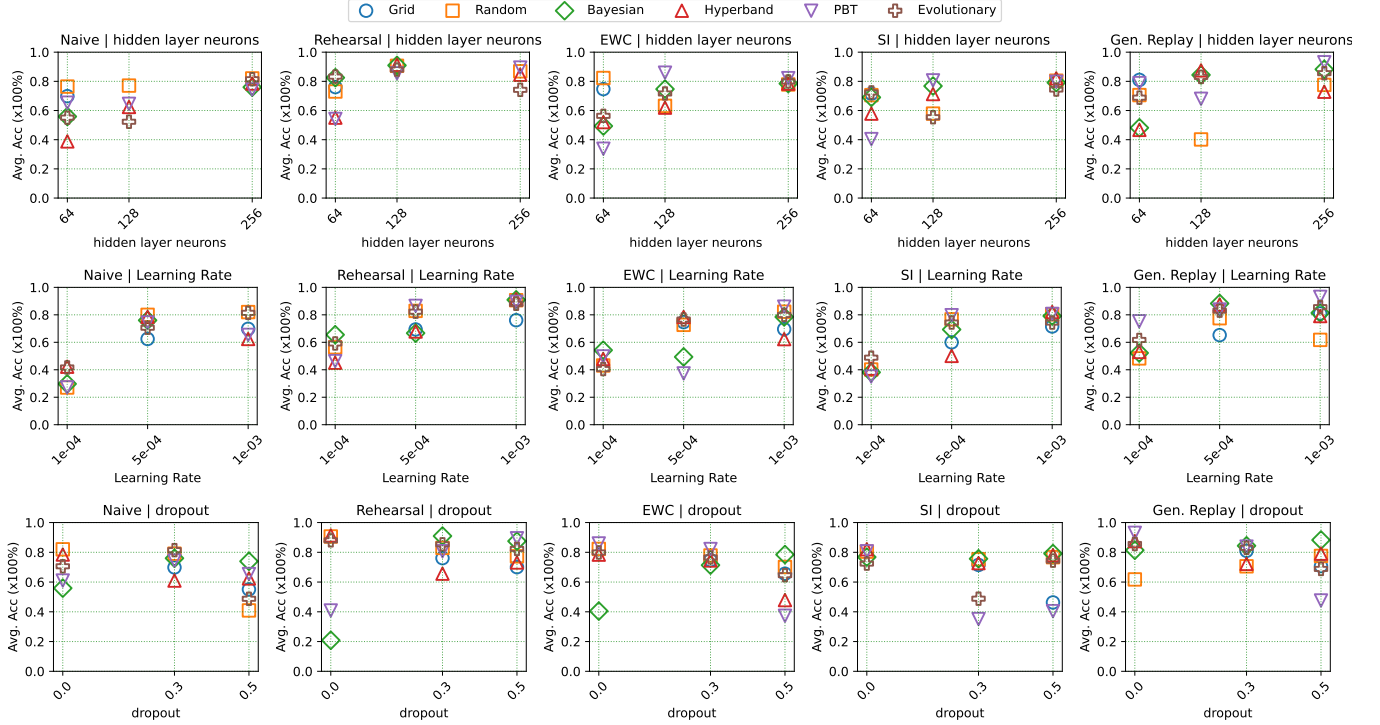


Fig. 1: Hyperparameter tuning for GCN static architecture for node classification. Accuracy represents the average taken across batches.

A. Theoretical complexity

For the message-passing GNNs used (GCN, GraphSAGE, GIN, GAT):

- **Per-epoch compute:** approximately $O(|E| \cdot d + |V| \cdot d^2)$, where d is the hidden dimension. The first term accounts for message aggregation across edges; the second for per-node MLP computations.
- **Memory:** activations and gradients scale as $O(|V| \cdot d + |E| \cdot d)$. Replay buffers add $O(B \cdot s)$ where B is buffer size and s is per-sample storage (features, labels, and any auxiliary metadata).

B. Practical strategies for large netlists

- **Hierarchical partitioning:** split very large netlists into module-level graphs and apply CIL per module or with a hierarchical aggregator to reduce per-step graph size.
- **Feature-level replay:** store node embeddings instead of raw node features to reduce buffer size and I/O overhead.
- **Sparse batching and neighbor sampling:** for extremely large graphs, use neighbor sampling or subgraph mini-batches to limit per-step compute.
- **Selective expansion:** in dynamic settings, expand only the output logits (or small heads) rather than entire layers to limit parameter growth.

IV. IMPROVED EXPLANATION OF FIGURES, TABLES, AND EXPERIMENTAL FLOW

This section clarifies how to read the main paper’s tables and figures and why certain presentation choices were made.

A. How to read Tables II–III (best configurations) ?

- **Avg. column:** arithmetic mean of per-batch accuracies reported for the configuration.
- **batch1–batch6:** per-batch accuracies in chronological order; these show how accuracy evolves as new classes are introduced.
- **Hyperparameter columns:** list the best hyperparameters found by the search strategy for that mechanism and architecture.
- **Static vs Dynamic:** static configuration rows are printed first (black), dynamic rows second (blue) within each GNN block in the PDF.
- **Dashes:** indicate not applicable or no observed improvement for that metric/cell.

B. Interpreting Table IV (catastrophic forgetting metrics)

Table IV links degradation/improvement to weight distance and mechanism-specific metrics. The intended interpretation:

- 1) **High degradation + large weight distance** $\Rightarrow$ parameter overwriting and instability.
- 2) **Low degradation + high mechanism metric (EWC/SI)** $\Rightarrow$ strong regularization preserved prior knowledge.
- 3) **Positive improvement in dynamic settings** can occur when expansion allows specialization without overwriting shared parameters.

The notebooks include aligned plots of per-batch accuracy and weight distance to visually support these interpretations.

C. Why tables were used ?

Tables provide precise, reproducible numeric values for each configuration and batch. To aid readability, the repository

notebooks also contain plotted summaries (bar charts and line plots) that aggregate and visualize the same data.

V. IMPLEMENTATION CLARIFICATIONS AND DEVELOPER NOTES

This section documents implementation details that affect interpretation and reproducibility.

A. SI behavior for link prediction

In the current pipeline SI importance accumulation was implemented to aggregate contributions from class-label gradients. Link prediction uses edge-level binary losses; the accumulation routine did not aggregate edge contributions correctly, resulting in zeroed SI importance values for link prediction runs. To avoid ambiguity, the repository marks SI link-prediction entries as implementation disabled.

How to enable SI for link prediction (developer note):

Below is a minimal code sketch that shows how to adapt SI accumulation to edge losses. This snippet is illustrative; adapt variable names to your codebase.

```
# Pseudocode: accumulate parameter contributions from edge
loss

# During training loop for a batch of edges:
optimizer.zero_grad()
pred = model(node_features, edge_index)
loss = edge_loss_fn(pred, edge_labels) # binary cross-
entropy over edges
loss.backward()

# For each parameter p:
for name, p in model.named_parameters():
    if p.grad is None: continue
    # accumulate path integral: delta_param * -grad
    delta = p.data - p_prev[name] # p_prev stored at
    batch start
    contribution = (-p.grad * delta).sum().item()
    si_omega[name] += contribution / (delta.pow(2).sum().
    item() + eps)

# After finishing batch, update p_prev for next
accumulation
for name, p in model.named_parameters():
    p_prev[name] = p.data.clone()
```

Listing 1: SI accumulation for edge losses (illustrative)

Key points:

- Accumulate contributions using the *edge loss* gradients rather than class logits gradients.
- Maintain per-parameter previous values (`p_prev`) and SI importance accumulators (`si_omega`).
- Normalize by squared parameter change to obtain stable importance estimates.

B. Reproducibility checklist

To reproduce reported results, follow these steps:

- 1) Run the parser notebook to generate graph datasets from ISCAS85 and EPFL netlists.
- 2) For a chosen GNN and mechanism, run the hyperparameter search notebook (or use the provided best configuration in Tables II–III).
- 3) Execute the incremental training notebook for the selected configuration; saved logs and plots will be produced in the folder.

- 4) For deterministic runs, set the random seeds for Python, NumPy, PyTorch, and DGL as shown in the notebooks.

Exact commands and environment specifications (Python version, package versions) are included in the repository README.

VI. PRACTICAL RELEVANCE AND USE CASES

This section expands on real-world scenarios where class-incremental learning for circuits is beneficial:

- **Engineering Change Orders (ECO):** incremental netlist updates introduce new components; CIL enables updating models without full retraining.
- **IP integration:** third-party blocks may add previously unseen gate types; CIL supports incremental adaptation.
- **Post-silicon updates and fault evolution:** labeling schemes or fault signatures can evolve over time; CIL preserves prior knowledge while learning new patterns.

For each scenario we discuss how replay, regularization, and dynamic expansion trade off compute, memory, and stability.

A. Tradeoffs of Mitigation Mechanisms Across Practical Scenarios

The goal is to give actionable guidance for selecting mechanisms given operational constraints. The summary is given in Table I and discussed next.

1) *Engineering Change Orders (ECO):* ECO workflows introduce small, frequent netlist updates (new components or modified connections). The primary operational goals are fast updates and minimal disruption to existing models.

a) *Rehearsal (Replay):* Rehearsal provides strong stability by replaying exemplars from prior data. In ECO settings it requires additional I/O and memory proportional to buffer size but yields high retention of prior knowledge. Rehearsal is recommended when modest extra memory is available and update latency from replay I/O is acceptable.

b) *Regularization (EWC / SI):* Regularization imposes negligible runtime overhead and minimal memory footprint (storing importance statistics). It is compute-light and fast to apply during incremental updates, making it attractive for low-latency ECO updates. However, regularization alone can underperform when new components substantially change representations.

c) *Dynamic Expansion:* Dynamic expansion preserves prior parameters by allocating new capacity for novel classes. It avoids replay I/O but increases model size and long-term memory usage. For ECOs that introduce only a few new classes, targeted expansion of small heads is effective; for frequent small updates, uncontrolled expansion can bloat the model.

2) *IP Integration:* Integrating third-party IP often introduces previously unseen gate types or modules. The priority is robust adaptation to new classes while preserving performance on legacy components.

TABLE I: High-level tradeoff summary for each mechanism across compute, memory, and stability.

Mechanism	Compute	Memory	Stability
Rehearsal (Replay)	Moderate to High; extra training on exemplars	Moderate to High; buffer stores exemplars	High; directly revisits prior data
EWC / SI (Regularization)	Low; small extra computations	Low; store importance stats only	Moderate; depends on importance accuracy
Dynamic Expansion	Low to Moderate per update; grows model size	High long-term; added parameters	High for new classes; risk of drift if overexpanded

a) Rehearsal (Replay): Replay is effective because exemplars can be chosen to represent both legacy and new IP interactions. Memory cost scales with exemplar count; compute overhead arises from mixing replayed samples during training. For IP integration, moderate buffer sizes targeted at representative subgraphs yield a good stability/compute balance.

b) Regularization (EWC / SI): Regularization is attractive when memory is constrained or when storing third-party data is restricted for IP licensing reasons. It maintains a small memory footprint but may struggle if the new IP induces large representational shifts; combining regularization with a small replay buffer often yields the best practical outcome.

c) Dynamic Expansion: Dynamic expansion enables specialization for IP-specific classes without overwriting shared representations. It is particularly useful when new IP requires distinct output heads or specialized parameters. The tradeoff is increased parameter count and potential maintenance complexity; selective expansion limited to small heads mitigates memory growth.

3) Post-Silicon Updates and Fault Evolution: Post-silicon environments see evolving labels and fault signatures over time. The objective is to incorporate new diagnostic patterns while retaining historical fault detection capability.

a) Rehearsal (Replay): Replay excels at preserving rare historical fault patterns by retaining exemplars. For fault evolution, prioritizing exemplars of rare or safety-critical cases is essential. Memory and I/O costs are justified when historical coverage is critical for reliability.

b) Regularization (EWC / SI): Regularization is useful for incremental adaptation when replay is infeasible (e.g., privacy or storage constraints). It provides a low-memory, low-compute option that reduces catastrophic forgetting, but may not capture rare fault modes unless importance estimates accurately reflect their criticality.

c) Dynamic Expansion: Dynamic expansion can isolate new fault signatures into specialized parameters, preserving legacy detectors. This approach increases model size but can improve stability for evolving fault classes. Combining expansion with targeted replay of critical historical faults yields robust detection with controlled memory growth.

B. Practical Recommendations

- **When low latency and low memory are required:** prefer regularization (EWC/SI) with careful hyperparameter tuning.
- **When stability and accuracy on legacy data are critical:** use rehearsal with a prioritized exemplar selection strategy; compress exemplars if storage is constrained.

- **When new classes require specialization:** apply selective dynamic expansion limited to small heads; combine with replay for critical legacy coverage.
- **Hybrid strategies:** combining small replay buffers with regularization or selective expansion often yields the best balance across compute, memory, and stability.

These guidelines help practitioners choose mechanisms that align with operational constraints for ECOs, IP integration, and post-silicon maintenance.

VII. ROADMAP FOR FUTURE ALGORITHMIC WORK

The benchmarking results identify concrete directions for algorithmic improvements:

- **Gate-criticality aware regularization:** weight penalties scaled by gate criticality (e.g., timing criticality or fan-out importance).
- **Topology-preserving replay:** select exemplars that preserve structural motifs (subgraph sampling with motif scoring).
- **Hybrid dynamic/static heads:** expand only when a new class exhibits representation drift beyond a threshold.
- **Edge-aware SI/EWC:** adapt importance accumulation to edge losses for link prediction tasks (developer note above).

VIII. MNIST TOY EXAMPLE

The repository contains a pedagogical FCN example on MNIST (folder `FCN_MNIST`). This static fully-connected network demonstrates CIL concepts (introducing digit classes incrementally) for readers unfamiliar with GNNs. The notebook includes step-by-step instructions and visualizations to ease the transition to circuit-level experiments.

IX. HYPERPARAMETER SEARCH STRATEGY RATIONALE

Six search strategies were used to balance exploration and exploitation:

- **Grid and Random:** baseline coverage of the search space.
- **Bayesian:** efficient exploitation of promising regions.
- **Hyperband:** resource-aware early stopping to evaluate many configurations quickly.
- **PBT:** population dynamics to evolve hyperparameters during training (helpful for regularization methods).
- **Evolutionary:** global search that can escape local optima but may converge slower.

Search is performed during the first batch to limit compute while still finding robust configurations; the best configuration is then used for full incremental training.

X. LIMITATIONS AND RESPONSIBLE USE

We explicitly state limitations:

- No exhaustive multi-seed statistical testing was performed; users should run additional seeds if formal hypothesis testing is required.
- SI importance for link prediction was disabled in the current implementation; enabling it requires the developer changes documented above.
- Scalability strategies (hierarchical partitioning, feature-level replay) are discussed but not experimentally validated in this work.

We encourage readers to use the provided logs and notebooks to validate findings on their own hardware and datasets.

REFERENCES

- [1] "Class incremental learning gnn circuits," <https://github.com/rkarn/GNNClass-Incremental-Learning>, 2026.